

합성 데이터는 이렇게 만들었다

생성 명세서 · v5 확정판 — 설정값 · 페르소나 · 노이즈 · 검증 게이트 전부

대상 — generator/config/synthetic-v2.yaml (602줄) + generator/generate.py (2,998줄)
산출물 — 20,000명 · 50개교 · 834학년 · 240문항 · 12개월 · **123,696,643행** · 40개 표 전부 · CSV 11GB
시드 — 20260803. 같은 시드 + 같은 config = 같은 데이터
이 문서가 아닌 것 — ~~왜~~ 이 값인지는 synthetic-data-decision-history.pdf (30문답 · 판본 6개), 검수 결과는 EDA-final-12m-v5.pdf (23쪽)

- 1 합성 데이터가 맡은 일 — 무엇을 대신하는가
- 2 생성 순서 11단계 — 순서에 제약이 있다
- 3 기간 · 규모 · 조직 — 학교 3계층과 학급 정원
- 4 성장 곡선 — 6국면 + 그 위에 곱해지는 계절성
- 5 생활 리듬 — 시간대 24값 · 크로노타입 · 요일
- 6 잔존 구간 6개와 복귀 유저
- 페르
- 7 소나 — 6유형 · 트레잇 10 · 노이즈 4겹 · 평균 보존 **핵심**
- 8 친구 그래프 — 허브 · 잠긴 유저 · 겨우 해금한 유저
- 9 질문과 투표 — 스코프 · 인기도 Zipf · 후보 패딩 · 비투표자
- 10 세션 경계 · 시간 예산 · 로그 노이즈
- 11 하트 경제 — 적립 · 소비 · 결제 · 고래
- 12 게시판 · 신고 · 학교 정보 · 텍스트
- 13 일부러 심은 이상치 3종
- 14 완료 판정 — 게이트 5종과 재현성
- 15 판본 이력 v1 → v5, 무엇이 깨졌나
- 16 아홉 번 뺀 함정 — "설정에 값이 있어도 코드가 안 읽으면 없는 것"
- 17 이 데이터로 할 수 없는 것 7가지

1. 합성 데이터가 맡은 일 — 무엇을 대신하는가

이 프로젝트에는 데이터가 **세 종류** 있고 각각 다른 일을 한다. 합성 데이터는 나머지 둘을 대신하지 않는다.

데이터	규모	맡은 일
구 Ping 서비스 MySQL final / hackle	4천만 행+ 677,085명	문제 발견 · 설계 근거. 실제 서비스의 실제 실패. 리포트 8편에서 구조적 결함 10가지를 뽑았고 결정 문서 16건 이 이 분석을 인용한다. 2023-03~2024-05 에서 멈춰 있어 파이프라인 원천으로는 못 쓴다 — 증분이 영원히 0행이라 DAG가 도는지 검증할 수 없다
실유저 Supabase	수십 명	파이프라인이 맞게 도는지. 계속 흐르므로 증분 · 워터마크 · <code>_deleted_at</code> · Airflow DAG가 실제로 검증된다

합성 v5 이 문서의 대상	1억 2,370만 행	파이프라인이 <i>버티는지</i> + 분석 연습 재료. 규모에서만 나오는 결함(파티션 한도 · 워터마크 정지)을 만나게 하고, 상관관계가 심어진 데이터를 준다
-------------------	----------------	---

왜 그냥 무작위 생성이 아닌가

각 행동을 독립적으로 무작위 생성하면 **상관관계가 없는 데이터**가 나온다. 그러면 "결제하는 사람은 어떤 사람인가" 같은 질문에 답이 안 나온다 — 답이 애초에 안 심겨 있기 때문이다.

이 생성기의 설계 전체가 한 문장으로 요약된다 — **사람을 먼저 만들고, 그 사람에게서 로그가 나오게 한다.** 7장 페르소나가 그 장치다.

2. 생성 순서 11단계 — 순서에 제약이 있다

`generate.py` 의 `run()` . 순서를 바꿀 수 없다 — 게시판이 신고보다 먼저여야 신고할 대상이 있고, 학교 정보는 유저가 있어야 열람 기록을 만들 수 있다.

조
직
생

1 성 — NEIS 실제 학교 5,724개 중 50개를 3계층으로 활성화, 학급 834개 유도

유
저
생

2 성 — 페르소나 · 트레잇 배정 → 트레잇 정규화 → 잔존 구간 배정 → 학교 · 학급 배정

친
구
그
래

3 프 — 같은 학교 80% / 같은 반 42%, 허브 3%, 잠긴 유저 7%

질
문
생

4 성 — 240문항, 스코프 CLASS 35 / SCHOOL 35 / GLOBAL 30

투
표
생

5 성 (가장 오래 걸림) — 세션 · 후보 4명 · 지목 · 인기도 Zipf · 패딩

하
트
원
장
계

6 산 — 적립 · 소비 · 결제를 시간순으로 재생, `balance_after` 확정

유
저 ·
세
션 ·
탈
퇴
기

7 록 — 접속 세션 병합, 로그 노이즈, 탈퇴 시점

게
시

8 판 — 글 · 댓글 · 좋아요 · 댓글좋아요

신
고 ·
제
재 ·
차

9 단 — 게시판/투표 대상이 이미 있어야 한다

추
천
거

10 절 — 실서비스에서 0행인 표를 채운다

학
교
정

11 보 — 급식 · 학사일정 · 공지 · 시간표 · 열람기록

그리고 마지막에 `write_persona_key()` 가 **정답지**를 따로 뽑는다 (7장).

3. 기간 · 규모 · 조직

3.1 기간 — 실유저 구간과 정확히 맞닿게

설정	값	이유
----	---	----

period.start_date	2025-07-29	웹앱 로그가 쌓이기 정확히 1년 전 ~ 쌓이기 전날 . 실측 <code>user_session</code> 최초가 2026-07-29 라, 합성 구간과 실유저 구간이 겹치지 않고 정확히 맞닿는다
period.end_date	2026-07-28	
scale.users	20,000	월별 · 이탈구간별 차이를 보려면 월 1,500명이 필요 → 12개월
scale.max_total_rows	100,000,000	안전판. 진짜 위험은 행수가 아니라 쿼리량 — Looker가 raw를 파티션 없이 직접 보면 무료 한도 1TB/월을 넘긴다
meta.seed	20260803	같은 시드 + 같은 config = 같은 데이터
meta.timezone_storage	UTC	저장은 UTC, 행동 패턴은 KST 기준으로 만든다. 안 그러면 대시보드에서 "밤에 몰린다"가 낮으로 보인다 (9시간 밀린다)
meta.timezone_behavior	Asia/Seoul	

3.2 학교 3계층 — 균등 분산은 학급 정원을 무너뜨린다

구 서비스 최대 결함이 **"5,951개 학교 중 딱 10곳에서만 서비스가 돌아간다"** 였다. 그걸 단되, 50개교를 균등하게 나누면 학교당 인원이 같아져 **"큰 학교와 작은 학교"** 라는 축이 사라진다.

계층	학교 수	학교당 인원	진입 시기와 역할
core	10	800 ~ 1,150	2025-09-01 ~ 09-20. 먼저 열린다 — 활동 대부분이 여기서 나온다
mid	15	380 ~ 700	2025-10-01 ~ 2026-03-31
tail	25	60 ~ 300	2026-02-01 ~ 07-01. 3월 봄학기 스파이크를 신규 학교가 받친다

학교 안에서는 `logistic` S자 곡선으로 퍼진다 (`adoption.mode: sequential`).

⚠️ 정원 합계는 유저 수보다 커야 한다

2026-08-04 수정. 옛 값은 기대 합계가 약 18,900이라 **20,000명 중 1,100명이 정원을 넘겨 배정됐다** (생성기가 경고를 찍는다). 12% 여유를 두고 넓혀 기대 합계 약 22,350.

3.3 학급 — 비율이 아니라 수가 기준이다

설정	값	이유
students_per_class	[20, 32]	이 값이 학급 수를 정한다 (학급 수 = 정원 ÷ 평균). 14~22로 두면 학급이 잘게 쪼개져 같은 반 친구가 4명이 안 되는 유저가 늘고 CLASS 스코프 투표가 GLOBAL로 물러선다. 실측 16.5는 "같은 반 앱 유저 수 " 였는데 그걸 학급 정원으로 쓰면 학급이 실제 학교보다 많아진다
max_students_per_class	35	절대 상한 . 넘으면 인기도가 학급 안 순위로 정해지는 구조에서 슬림이 커진다 (77명일 때 실제로 그랬다)
middle_ratio	0.55	실측: 중학교 3,290 / 고교 2,661
master_source	neis	300개 합성 대신 DB에 있는 NEIS 실제 5,724개 를 쓴다

4. 성장 곡선 — 6국면 + 그 위에 곱해지는 계절성

v1은 첫 2주에 60%가 가입하고 그 뒤 단조 우하향이었다. 실패도 저점도 회복도 없으니 코호트를 갈라볼 것이 없다 — 어느 달에 들어와도 같은 곡선을 그린다. v2는 "저점 → 회복 → 성장"이었으나, 학교 기반 서비스에는 봄학기 스파이크 뒤 서서히 감소가 맞다고 판단해 `spring_spike` 로 바꿨다.

국면	기간	가입 비중	활동 배수	서사
launch_spike	2025-09-01 ~ 09-30	0.35	1.00	런칭. v1은 한 달에 93.9% — 완화 재현
decline	2025-10-01 ~ 12-31	0.15	0.45	관심이 식는다
trough	2026-01-01 ~ 02-28	0.05	0.18	겨울방학 최저점. 1월 한 달 → 1~2월 두 달로 늘렸다
spring_spike	2026-03-01 ~ 03-31	0.20	0.80	봄학기. 9월의 57% 규모 — 두 번째지만 더 작다
slow_fade_1	2026-04-01 ~ 05-31	0.15	0.60	스파이크 뒤 완만한 하강
slow_fade_2	2026-06-01 ~ 07-31	0.10	0.42	기말 · 여름방학이 겹치며 더 내려간다

검수 결과 — 저점 대비 회복 4.2배로 목표(2.5배)를 넘겼다.

4.1 계절성 — 국면 위에 곱해진다

구간	기간	배수
여름방학	2025-08-01 ~ 08-25	0.35
겨울방학	2025-12-24 ~ 12-31	0.55
시험기간 (중간 · 기말 각 2주)	2025-10-13 · 2025-12-08 · 2026-04-20 · 2026-06-22	0.40

⚠ 이 블록이 실제로 두 번 깨졌다

① 절이 밖으로 밀려나 통째로 무시됐다. 2026-08-04에 `behavior` 블록을 넣다가 `seasonality` 가 `growth` 밖으로 나가 `None` 이 됐고 시험기간 배수가 전혀 안 걸렸다 (평시의 108%). 겨울방학이 28%로 나온 건 국면 배수(`trough` 0.18) 덕이지 이 설정 덕이 아니었다.

② 배수가 두 번 곱해졌다. 겨울방학 범위가 2026-02-05까지라 `trough` 국면(0.18)과 겹쳐 $0.18 \times 0.30 = 0.054$ — 겨울 활동이 평시의 9%로 내려앉았다 (목표 30~45%). 범위를 12월 말로 좁혀 국면이 안 덮는 구간만 맡게 했다.

둘 다 오류가 안 났다. 데이터를 다 만들어 분포를 봐야 드러난다.

seasonality_affects_volume: true

false 면 배수가 세션의 날짜만 옮긴다. 그런데 유저 대다수는 활동 창이 짧아 (same_day_only 26% · within_week 32%) 옮길 곳이 없다. 그래서 v2에서 겨울방학이 0.30 설정인데 실측 0.66배에 그쳤다. true 면 실제로 덜어낸다.

5. 생활 리듬 — 시간대 24값 · 크로노타입 · 요일

⚠ 이 블록은 Q23에서 정해 놓고 한 번도 구현되지 않았다

v3 실측이 0~23시 전부 4.1~4.2% — 완전 균등이었다. Q23이 그 이유를 이렇게 적어 두었다: "Looker 대시보드의 시간대 히트맵이 밋밋하면 시각화할 게 없다." 설정에 값은 있었지만 읽는 코드가 없었다 (16장 참조).

5.1 시간대 가중치 — Q23의 모양을 뒤집었다

Q23은 점심(12~13시)을 최대 봉우리로 봤으나, 한국 학생은 점심에 폰을 오래 못 쓰고 소셜 투표는 하교 후 ~ 취침 전에 몰린다. 최대 봉우리를 22~23시로 옮겼다. 상대 가중치이며 합이 1일 필요는 없다.

	0	1	2	3	4	5	6	7	8	9	10	11
평 일	.15	.12	.10	.08	.08	.10	.18	.35	.38	.34	.32	.34
주 말	.22	.18	.14	.10	.09	.10	.15	.30	.35	.38	.40	.45
	12	13	14	15	16	17	18	19	20	21	22	23
평 일	.62	.58	.28	.28	.32	.85	.88	.90	.92	.95	1.00	.98
주 말	.60	.62	.60	.60	.62	.65	.70	.80	.85	.88	.90	.85

평일은 수업시간(14~16시)에 폭 꺼지고 하교 후 급등하는 이중 봉우리, 주말은 낮부터 완만하게 올라가는 평평한 모양이다.

5.2 크로노타입 — 전원이 같은 곡선을 따르면 안 된다

페르소나와 같은 이유다 — 모두가 22시에 몰려 있으면 "심야형 유저"라는 분석 대상이 사라진다. 유형마다 봉우리를 옮기고, 개인 편차로 한 번 더 흔든다.

유형	비율	shift	flatten	뜻
normal	0.65	0	0.0	위 곡선 그대로
night	0.15	+3	0.0	봉우리가 3시간 뒤로 (22시 → 01시)
morning	0.10	-6	0.0	6시간 앞으로 (22시 → 16시)
allday	0.10	0	0.8	곡선이 평평 — 시간 구분이 약하다

여기에 흐리는 장치 3개가 더 붙는다.

chronotype_mix_ratio	0.12	두 유형을 6:4로 섞는 유저
chronotype_none_ratio	0.04	유형 없이 순수 무작위인 유저
chronotype_jitter	1.5	개인마다 봉우리가 ±1.5시간 흔들린다

5.3 요일 — 구 서비스 실측을 따라간다

구 Ping의 출석 로그에서 뽑은 실측 패턴이다. **우하향이 아니라 V자** — 월 · 화가 높고 **금요일이 최저**, 주말에 회복한다.

요일	월	화	수	목	금	토	일
가 중 치	1.00	0.97	0.73	0.70	0.66	0.85	0.90

dow_gamma: 1.6 — **진폭 보정 지수**. 가중치를 그대로 쓰면 계절 · 시간대 배수와 섞이면서 진폭이 늘린다 (실측 최저 0.76). 1.6을 걸면 실측이 원본(0.66)에 맞는다.

⚠ **v4에는 요일 효과가 아예 없었다 (변동계수 0.024)**

설정에 weekend_multiplier: 0.55 가 있었지만 읽는 코드가 없었다. 그래서 v5는 상수를 코드 옆에 두고 (DOW_WEIGHTS), 설정으로는 켜고 끄기만 한다 (use_dow_weights). 값이 설정에만 있으면 또 조용히 죽는다.

6. 잔존 구간 6개와 복귀 유저

활동 강도는 잔존 구간에서 갈린다. 실측에서 **의도적으로 벗어난** 곳이 하나 있다.

구간	설정	v1 실측	월 세션 수	뜻
no_activity	0.041	0.041	0	앱을 연 적이 없다
same_day_only	0.260	0.284	1 ~ 4	가입 당일만
within_week	0.320	0.355	4 ~ 20	1~6일
within_month	0.240	0.278	25 ~ 70	7~29일
within_quarter	0.100	— 신설	50 ~ 110	30~89일. 실측에 없던 구간을 넣었다
long_term	0.039	0.042	100 ~ 200	90일+ 활성 유저. 하루 3~7회

왜 실측을 버렸나

실측대로 두면 within_month (최대 29일) 위가 곧바로 long_term 이라 **"한 달과 장기 사이"**가 비어 있다. 그 결과 12개월 코호트에서 M1→M2가 **26% → 4%** 로 절벽이 됐다. 잔존 곡선의 *모양 자체*를 분석할 수 없게 되므로 within_quarter 를 넣고 나머지를 조금씩 덜었다.

6.1 복귀 유저 — 휴면했다 돌아오는 사람

코호트 분석의 재료다. reactivation_rate: 0.09, 창은 2026-02-01 ~ 07-31, 두 번째 활동 창 길이는 [7, 45] 일 (첫 창보다 짧다).

3월에만 물면 "복귀 = 3월"이 되어 코호트를 갈라볼 것이 없다. 그래서 월별 가중치를 나눈다 — 2026-03 3.0 · 04 1.5 · 02 1.0 · 05 1.0 · 06 0.8 · 07 0.6.

6.2 탈퇴

withdrawal_rate	0.105	실측: 가입자의 10.5%
withdrawal_free_text_rate	0.35	자유 서술을 남기는 비율
gender_ratio_f	0.586	실측: 여 396,665 / 남 280,418

사유 분포 — NO_FRIENDS 0.30 · BORING_CONTENT 0.27 · NOT_USING 0.20 · OTHER 0.12 · TOO_MANY_BUGS 0.06 · PRIVACY 0.05. 구 서비스 실측(친구부족 20.4% > 콘텐츠 18.6% > 버그 2.9%)의 순서를 따른다.

7. 페르소나 — 이 생성기의 핵심 장치

페르소나는 분류가 아니라 생성 편의다

유저를 6종으로 나누는 것이 목적이 아니다. 목적은 행동 트레잇들이 서로 묶여서 움직이게 하는 것이다. 그 전에는 글쓰기와 투표가 독립이라 "글도 쓰고 투표도 많이 하는 사람"이 우연으로만 생겼다.

7.1 트레잇 10종 — 행동의 원인

배수는 상대값이다. 절대 확률이 아니라 "모집단 평균의 몇 배인가".

트레잇	거는 곳	트레잇	거는 곳
vote_freq	투표 빈도 · 세션 수	post	글쓰기
read	받은 투표 열람	comment	댓글
hint	힌트 구매	pay	하트 결제
reply	1회성 답장	reported	신고당함
friends	친구 수	retain	잔존 (구간 배정에 쓰인다)

7.2 6유형과 배수표

유형	비중	vote	read	hint	reply	friend	post	comm	pay	rept	retain
관망형	0.30	0.4	1.2	0.5	0.4	0.7	0.3	0.4	0.4	0.6	0.5
투표형	0.28	1.5	1.0	1.1	0.8	0.9	0.4	0.6	1.0	0.8	1.3
소셜형	0.15	0.9	1.0	0.9	1.5	2.0	0.8	1.0	0.8	1.0	1.3
게시판형	0.12	0.6	0.9	0.7	1.0	0.9	2.6	2.2	0.7	1.2	1.2
고관여형	0.12	2.2	1.3	1.9	1.8	1.5	1.8	1.7	2.6	1.0	2.6
악성형	0.03	1.2	0.8	1.3	1.4	0.7	1.4	1.3	0.9	8.0	0.9

7.3 노이즈 4겹 — 6덩어리가 또렷하게 남으면 실패다

코드 주석에 적힌 실패 조건: "결과 데이터에 6덩어리가 또렷하게 남으면 실패다. 분석자가 클러스터링 해서 우리가 넣은 6개를 그대로 되찾는 데이터는 발견할 것이 없다."

장치	비율	하는 일
혼합 MIX_RATIO	0.25	두 유형을 6:4로 섞어 경계를 없앤다 → 라벨이 "관망형+고관여형"
교차 CROSS_RATIO	0.08	트레잇 하나만 다른 유형에서 통째로 가져온다 → "관망형인데 댓글만 게시판형인 사람"이 여기서 나온다. 라벨이 "관망형←comment←게시판형"
무배정 FREE_RATIO	0.04	유형 자체가 없다. 전 트레잇 독립 무작위 <code>uniform(0.3, 2.2)</code>
개인 편차 TRAIT_SIGMA	σ 0.55	로그정규분포를 트레잇마다 따로 곱한다

⚠️ **마지막이 특히 중요하다** — 한 배수로 전부 흔들면 안 된다

유저 하나에 배수 하나를 곱하면 **유형 내부의 순서가 그대로**라 클러스터가 살아남는다. 그래서 트레잇마다 **따로** 흔들다.

`vote_freq` (σ 0.90)와 `retain` (σ 0.80)은 σ 를 더 크게 준다 — 세션 수가 이 둘의 곱이라 편차가 작으면 유형 간 차이가 **63배**까지 벌어져 **"관망형인데 자주 오는 사람"**이 한 명도 안 나온다. 실제로 그랬다.

7.4 평균을 지키는 두 장치

개인차를 걸면 **전체 평균이 어긋난다**. 힘들게 맞춰 둔 `read_rate` · `author_ratio` 같은 값이 조용히 틀어진다 — 오류는 안 난다. 두 장치가 그걸 막는다.

① `_normalize_traits()` — 모집단 평균을 정확히 1.0으로

해석적으로 계산하지 않고 **실측 평균으로 나눈다**. 개인 편차의 분포가 무엇이든 정확히 보존되기 때문이다.

② `pchance()` — 오즈비 변환의 손실을 이분법으로 되돌린다

확률에 배수를 그냥 곱하면 1을 넘으므로 **오즈비**로 건다 ($p=0.55$, $m=2 \rightarrow 0.71$, 1.10이 아니다). 그런데 **트레잇 평균이 1.0**이어도 오즈비 변환을 거치면 **평균이 내려간다** — 변환이 블록해서다.

실제로 열람률이 55.5% → 39.2%로 땀다. 설정값을 손으로 올려 맞추면 트레잇 분포가 바뀔 때마다 다시 틀어지므로, **보정 계수를 이분법으로 50회 풀어** 캐시한다.

한 줄로 — 개인은 다르게, 전체는 목표값 그대로.

7.5 정답지 — `data/personas.json`

생성이 끝나면 `write_persona_key()` 가 **5MB · 20,000명**분의 정답지를 따로 뽑는다. 파일 첫 줄이 `"note": "생성기 내부 상태. 분석자에게 주지 말 것."` 이다.

```
"2": {
  "persona": "고관여형",
  "tier": "within_quarter",
  "traits": { "vote_freq": 0.6593, "read": 1.2526, "hint": 1.2987,
    "reply": 0.8613, "friends": 0.4927, "post": 1.0816,
    "comment": 1.2519, "pay": 5.6527, "reported": 0.6038,
    "retain": 1.8533 }
}
```

DB에도 CSV에도 들어가지 않는다. 정답을 보고 시작하면 클러스터링에서 발견할 것이 없어지기 때문이다.

현재 실제 용도 — "맞았나"가 아니라 "너무 티나지 않나"

`docstring`이 정확하다: *"검증(유형이 너무 또렷하지 않은가)에만 쓴다."*

즉 지금은 내가 심은 게 너무 쉽게 되찾아지지 않는지 확인하는 용도다. 분석 결과를 채점하는 데도 쓸 수 있지만, 그건 아직 하지 않았다.

8. 친구 그래프

설정	값	이유
per_user / median	[5, 60] / 30	기본 분포
same_school_ratio	0.80	실측 79.6%
same_class_ratio	0.42	실측 23.7%에서 의도적으로 올렸다. 이건 뽑는 비율이라 학급을 키워도 같은 반 친구 수는 안 늘어난다 (학급 15→26명으로 키웠는데 같은 반 0~3명이 25.7% → 26.9%로 오히려 늘었다). CLASS 스코프는 같은 반 친구가 4명 있어야 성립하므로 기준은 비율이 아니라 수다. 목표: 0~3명 구간을 15% 아래로 → 실측 14.7% 통과
request_accept_rate	0.751	실측 그대로. 수락 75.1% / 대기 23.0% / 거절 1.9%
request_pending_rate	0.230	
request_reject_rate	0.019	
unfriend_rate	0.06	관계의 6%가 끊긴다. ended_at 이 찍히고 행은 남는다

8.1 허브 유저 — 인기도 집종의 절반이 여기서 나왔다

hub_ratio: 0.03 · hub_per_user: [80, 240]. 친구가 많으면 GLOBAL 후보 풀이 커져 투표 경험 자체가 달라진다. "친구가 많은 유저는 다르게 행동하는가"를 물어려면 꼬리가 충분히 길어야 한다.

원래 200~700이었다. 친구가 162명이면 162명의 후보 목록에 오르고 39명이면 39번만 오른다 — 노출이 4배다. zipf_alpha 는 후보 4명 중 누가 뽑히느냐만 정하므로 아무리 낮춰도 이 부분이 안 줄어든다 (실측: 0.80→52.6% / 0.45→50.7%, 바닥이 약 48~49%). 꼬리를 유지하되 극단을 줄였다 — 중앙값 30 대비 4~12배면 여전히 명백한 허브다.

8.2 게이트 주변의 두 구간

설정	값	왜 필요한가
locked_user_ratio locked_hard	0.07 true	친구 5명 미만이라 서비스를 못 여는 유저. 남이 친구로 추가해도 고립이 유지돼야 한다 — 양방향 유입을 막지 않으면 이 비율이 무의미해진다 (실제로 그랬다). 목표 해금률 90~95% → 실측 92.6% 통과
barely_unlocked_ratio barely_unlocked_target barely_exclude_prob	0.08 [5, 9] 0.75	딱 해금한 유저 — 친구가 5~9명에 머무는 사람. ⚠ 이 구간이 v3 데이터에 한 명도 없었다. 친구가 양방향이라 게이트를 넘는 순간 남들이 계속 추가해 10명 이상으로 끌려 올라갔기 때문이다. locked_hard 는 5명 미만만 남의 풀에서 뺐고 그 위는 아무 장치가 없었다. "겨우 열었지만 거의 안 쓰는 사람"이라는 상태를 만들려면 이 구간이 필요하다

rejected_recommendation — 유저의 15%가 1~5회 "안 볼래"를 누른다. 실서비스에서 0행인 표를 채우는 곳이다.

9. 질문과 투표 — 서비스의 심장

9.1 질문

count	240	문항 수
scope_ratio	35 / 35 / 30	CLASS / SCHOOL / GLOBAL
include_appearance appearance_ratio	true 0.06	외모 · 신체 질문을 컷다 — 신고율을 재보기 위해서다. 민감한 질문이 실제로 얼마나 신고되는지 모르면 <code>is_sensitive</code> 플래그가 데이터에서 아무 의미도 갖지 못한다. 칭찬 방향만 쓰고 비하로 읽힐 문장은 넣지 않는다. 240개 중 16개
sensitive_ratio	0.18	외모 포함으로 12% → 18%
user_submitted_ratio	0.15	유저 제출 질문 몫
request_count / approve	320 / 0.45	질문 요청과 승인을

9.2 투표 세션

items_per_session	10	한 세션에 10문항
complete_rate	0.963	실측: 문항 노출 → 투표 완료 96.3%
session_abandon_rate abandon_early_share	0.22 0.60	끝까지 안 채우고 나가는 비율. 없으면 퍼널 분석이 성립하지 않는다. 이탈의 60%는 1~3번째 문항에서
padded_item_ratio	0.18	후보가 모자라는 상황을 실제로 만든다 — 꼬리 학교에서. 채운 수는 <code>vote_item.padded_count</code> 에 남는다
recent_exposure_days	7	최근 후보로 나온 사람은 다시 안 뽑는다 (풀이 모자라면 푼다)
shuffle_rate / ad_complete	0.18 / 0.92	셔플 비율과 광고 완료율
power_user_ratio power_user_multiplier	0.06 6	소수가 활동 대부분을 만든다 (v1 실측 0.7%보다 완화)

9.3 "해금은 했는데 투표를 한 번도 안 하는 유저"

v3에서 해금자의 14.6%가 투표 0회였는데, 그건 계절성 숙아내기의 부작용이지 설계가 아니었다 (세션이 1개뿐인 유저는 그 하나가 숙이면 0이 된다). 사고와 의도가 섞이면 어느 쪽인지 구분할 수 없으므로, 숙아내기에는 최소 1개를 보장하고 비투표는 여기서 명시적으로 만든다.

친구 수	5	6~9	10~19	20~39	40+
투표 0회 확률	0.40	0.20	0.07	0.02	0.005

0이 아니다 — 친구 40명인데 안 하는 사람도 소수 존재해야 "친구 수가 곧 활동"이라는 동어반복을 피한다.

그리고 `never_vote_trait_weight: true` — 확률을 그대로 쓰면 "친구 5명 = 40%"라는 **규칙**이 된다.
`vote_freq` 트레잇으로 나눠 사람마다 다르게 만든다 (페르소나와 같은 방식).

9.4 인기도 — 분석의 재미가 여기서 결정된다

지목이 **Zipf 분포**로 쏠린다. 인기도는 **학급 안에서** 정해진다 — *반의 스타 ≠ 전교의 스타*. 목표는 **상위 10%가 전체 지목의 49%**.

top10_share	0.49 목표	원래 0.45였으나 도달할 수 없는 값 임이 실측으로 드러났다
zipf_alpha	1.05	클수록 쏠린다
scope	class	학급 안에서 순위가 정해진다
never_picked_ratio	0.04	아무에게도 지목받지 못하는 사람이 있어야 한다 . 전원이 최소 1표는 받는 데이터는 "인기 없음"이라는 상태 자체를 지워버린다

⚠ **alpha가 0.45 → 1.05로 오른 이유 — 목표가 바뀐 게 아니라 바닥이 내려갔다**

v4에서 0.45로도 48.8%가 나온 것은 인기도가 아니라 **활동 불균형** 덕이었다 — 하루 2,243세션 · **406.8 시간** 같은 불가능한 꼬리가 쏠림을 떠받치고 있었다. v5가 그 꼬리를 시간 예산으로 잘라내자 **43.7%로 주저앉았다**.

이제 쏠림을 **매력도로** 만들어야 해서 alpha를 올렸다 — **이쪽이 원래 의도한 경로다**. 활동 불균형이 만든 쏠림은 인기도가 아니다.

v5 실측 곡선 (4,000명 프로브): 0.45 → 43.7% · 0.95 → 47.9% · 1.30 → 50.9%. 기울기 약 8.6%p / alpha 1.0 → 목표 49%는 1.05 근처. **최종 실측 49.8% 통과**.

`gender_homophily` — `romance_cross_gender: 0.78` · `friendship_same_gender: 0.65`. **성별이 힌트로 팔리는 구조의 근거다**.

10. 세션 경계 · 시간 예산 · 로그 노이즈

10.1 세션 경계 — 두 표가 따로 놀았다

유저의 직접 행동(투표 · 힌트 · 글 · 댓글 · 좋아요)은 접속 세션 안에서만 일어나야 한다. v4는 두 표를 따로 만들어서 투표 세션의 3.84%만 접속 세션 창 안에 있었다. "앱을 열지도 않았는데 투표한" 로그가 96%였다는 뜻이다.

설정	값	뜻
idle_gap_minutes	30	이만큼 비면 새 세션. 실서비스 touch_session() 과 같은 값
votes_per_app_session	[1, 3]	한 번 열어서 투표 라운드를 몇 번 도는가
browse_only_ratio	0.35	아무것도 안 하고 닫는 세션의 몫
session_tail_minutes	[0.5, 6]	마지막 행동 뒤 얼마나 더 머무는가
max_vote_session_minutes	90	투표 세션 길이 상한 — 말이 안 되는 값을 막는다
browse_minutes	[1, 40]	열어만 보는 세션의 길이. v5에서 로그정규로 바꿨다 — v4는 randint 라 2~25분이 완전 균등이었다 (균등도 1.042)
per_active_day	[1, 4]	활동한 날의 세션 수

platform_ratio — WEB 0.82 / ANDROID 0.11 / IOS 0.07. v1은 iOS 68 / AOS 32였지만 ping-v2는 웹앱이라 뒤집었다. 모바일 브라우저 UA로 구분되는 몫이다.

10.2 하루 상한 — 개수가 아니라 시간 예산으로

⚠ v4에는 하루에 406.8시간 투표한 유저-일이 있었다

하루는 24시간이다. 문제는 "많이 한다"가 아니라 물리적으로 불가능하다는 것이었다.

개수 상한(12회)을 걸어 봤더니 헤비유저가 통째로 사라져 행이 반토막 나고 지목 쓸림이 39%로 주저앉았다 — 결함을 고친 게 아니라 분포를 죽인 것이다.

그래서 시간 예산으로 바꿨다. 사람이 하루에 낼 수 있는 시간만 제약하고, 그 안에서 몇 번을 하든 내버려 둔다. 상한은 개수가 아니라 시간에서 유도된다.

max_vote_hours_per_day	6.0	하루에 투표에 쓸 수 있는 시간
max_active_hours_per_day	10.0	하루에 앱에 머무는 시간 (투표 + 열람)
est_vote_session_minutes	7	예산을 세션 수로 환산할 때 쓰는 평균 (실측 6.5분)

이 변경이 v4의 1억 8,922만 행을 v5의 1억 2,370만 행으로 줄였다 — 줄어든 것이 아니라 없어야 했던 것이 빠진 것이다.

10.3 로그 노이즈 — 실제 운영 로그의 지저분함

noise.duplicate_rate	0.004	같은 세션 행이 한 번 더 쌓인다 (재시도 · 중복 전송)
noise.duplicate_max	2	최대 몇 벌까지
noise.late_close_rate	0.002	종료 기록이 늦게 도착해 끝이 늘어난다
noise.late_close_minutes	[5, 90]	얼마나 늦는가

이 0.4%가 분석 연습의 재료다. 중복 로그를 안 걸러내면 체류시간이 부풀려진다 — "중복 로그 정제 연습"이 v5에서 열린 9가지 분석 중 하나인 이유다.

11. 하트 경제

⚠ v1 누진요금은 현행 스키마와 맞지 않는다

v1의 [200, 300, 500, 1000] 누진요금은 W14에서 폐기됐다. 현행은 골라 사는 5+1 구조다. v1 값을 쓰면 소비량이 10배로 부풀어 경제 전체가 어긋난다.

11.1 소비 — 힌트

hint_basic_cost	20	기본 5종 각 20하트
hint_fullname_cost	100	이름 공개. 기본 3개 이상 연 뒤에만
hint_gender_by_ad_ratio	0.30	성별 힌트를 광고로 여는 비율 → heart_cost = 0 인 행이 생긴다. 정합성 검사 4번이 이걸 전제로 고쳐져 있다
hint_buyer_ratio	0.86	힌트는 건별 확률이 아니라 사람의 성향으로 산다. 이걸 나누기 전에는 모두에게 같은 확률을 적용해서 힌트 횟수가 받은 투표 수에만 따라갔다 — "힌트를 잘 사는 사람"이 없었다. 목표 70~80% → 실측 76.1% 통과
hint_appetite_range	[0.05, 0.85]	

11.2 적립

signup_grant	300	실측: point 최빈값 300
vote_reward_weights	2~10하트	2하트 40% · 3하트 30% · 4하트 20% · 5하트 5% · 나머지 5%
reward_both_sides	true	실측: 투표자와 지목당한 쪽 양쪽 지급
ad_reward	0	셔플 광고에는 보상이 없다. 광고는 오직 셔플을 위해 본다
daily_earn_cap	100	하루 적립 상한. 파워유저가 무한히 쌓는 것을 막는다
stop_voting_at_cap	false	상한에 걸려도 투표는 계속한다 — 상한은 하트를 막지 행동을 막지 않는다

실측을 버린 곳이다. v1 실측 보상은 5~15(평균 10)였다. 그대로 두면 적립이 소비의 **8배**가 된다 — v1은 힌트가 200~1,000하트라 균형이 맞았지만 W14에서 **20하트로 내렸기 때문이다**. 근거는 힌트 단가 변경이다.

11.3 결제

topup_rate	0.18	결제 전환율. 실측 8.74%보다 높게 잡는다 (분석 표본 확보)
topup_repeat	[1, 4]	일반 결제자의 결제 횟수
whale_ratio	0.04	헤비과금러가 존재해야 한다. 결제자의 4%가 결제액의 절반 가까이를 만든다
whale_repeat	[12, 40]	
product_weights	55 / 25 / 14 / 6	일반 결제자는 소액 편중 (최저가 → 최고가)
whale_product_weights	5 / 15 / 30 / 50	헤비과금러는 고액 편중
use_stub_prefix	false	합성은 실결제로. MVP-STUB- 는 실유저 쪽에만 붙는다
target_end_balance_under_100	0.60	완만한 부족. 하트가 남아돌면 결제할 근거가 없다
ledger_consistency	strict	하트 · 원장 불일치는 재현하지 않는다. 예외 없음. <code>balance == SUM(heart_transaction)</code> , 생성 후 자체 검사

12. 게시판 · 신고 · 학교 정보 · 텍스트

12.1 게시판

author_ratio / commenter_ratio	0.40 / 0.70	활성 유저 중 글쓴이 / 댓글쓴이 비율
posts_per_author	[1, 25]	상위 5%가 글의 40% (author_pareto_top5_share)
comments_per_post_mean	2.5	댓글 0개인 글이 30% (zero_comment_post_ratio)
post_likes_per_post_mean	4.0	글당 좋아요
liker_ratio	0.60	좋아요를 누르는 사람의 비율. v4는 이 값이 없어서 활성 유저 전체가 후보가 됐고 결과적으로 95.9%가 늘었다
viral_post_ratio viral_post_likes	0.02 [80, 400]	떡상 글이 존재해야 한다
cross_school	false	학교 경계 안에서만

12.2 신고 · 제재 · 차단

reports_per_1000_vote_items	4	신고 밀도
reviewed_rate	0.70	30%는 PENDING으로 고인다 — 버그가 아니라 운영 현실이다
actioned_rate	0.25	검토된 것 중 제재로 이어진 비율
sensitive_report_multiplier	3.5	민감 질문이 몇 배 더 신고되는가. 목표 2배+ → 실측 3.8배 통과
target_type_mix	42/26/22/10	USER / POST / COMMENT / QUESTION. 한 종류만 나오면 신고 화면 분석이 안 된다
block.user_ratio / per_user	0.03 / [1,4]	차단

v2는 신고 대상 질문을 완전 균등하게 골랐다. 그래서 APPEARANCE(0.439)와 HUMOR(0.379)가 사실상 같았고 is_sensitive 플래그가 데이터에서 아무 의미도 갖지 못했다. 외모 카테고리를 연 목적이 신고율 측정이었으므로 이건 실패다. 이 배수가 그 신호를 만든다.

report_target ENUM은 USER · QUESTION · POST · COMMENT 넷뿐이다. 투표 신고와 답장 신고는 **USER** 신고로 들어간다 — 신고 대상이 콘텐츠가 아니라 사람이기 때문이다.

12.3 학교 정보 6표

source: replicate_existing — **NEIS API를 새로 부르지 않는다.** 지금 DB에 있는 **20개** 학교분 실데이터를 본떠 **50개** 학교분으로 늘린다 (문장 구조 · 메뉴 조합을 섞어 재조합).

급식 평일만 · 메뉴 4~7개 · 학사일정 학교당 월 2~4건 · 시간표 주 30교시 · 공지 학교당 월 2~4건 · 공지 열람률 20~60%.

12.4 텍스트

mode	corpus	템플릿 + 미리 만들어 커밋한 코퍼스. LLM을 생성 경로에 두지 않는다 (재현성 때문)
nickname_style	realname	닉네임은 별명이 아니라 본명처럼 만든다
forbid.phone_email_patterns	true	전화번호 · 이메일 문자열은 만들지 않는다
forbid.real_school_names	false	학교명은 NEIS 실명 그대로
forbid.profanity	false	신고 사유에 실제 욕설이 있어도 괜찮다 — 모더레이션 분석의 재료다

13. 일부러 심은 이상치 3종

탐지 연습용이다. 전부 정상 스키마 안에서 표현된다 — 제약 위반이 아니라 통계적 이상치여야 한다. 정답지는 `generator/output/anomalies.json`에 따로 나가고, 페르소나와 마찬가지로 분석자에게 주지 않는다.

패턴	유저 수	무엇
<code>vote_abuser</code>	5	하루 200회 투표
<code>night_only</code>	12	새벽 3~5시에만 활동
<code>repeat_targeter</code>	20	지목의 85%가 한 사람에게 집중

14. 완료 판정 — 게이트 5종과 재현성

"다 만들었다"의 정의다. 하나라도 실패하면 그 판본은 확정하지 않는다.

게이트	판정 기준
정합성 검사	<code>qa/checks/integrity.sql</code> 17종 위반 0
컬럼 커버리지	342컬럼 전부가 비-NULL 1% 이상. 화이트리스트는 사유 없이 넣지 않는다 — 늘어나는 만큼 이 검사가 쓸모없어진다
표 커버리지	40표 전부 1행 이상
충분 적재 실증	1차 + 2차 배치 행수 대조
분포 리포트	사람이 눈으로 본다 — 기계는 이상한 곡선을 못 막는다

14.1 화이트리스트 — 1%를 못 넘는 것이 정상인 컬럼

컬럼	사유
<code>app_user.auth_user_id</code>	설계상 반드시 NULL. 합성 유저는 Supabase 익명 계정이 없고, 이 NULL 자체가 실유저/합성을 가르는 신호다
<code>heart_transaction.purchase_id</code>	결제 거래는 전체 거래의 0.1%대
<code>heart_transaction.admin_id</code>	운영자 보정은 더 드물다
<code>heart_purchase.failure_reason</code>	실측 결제 실패율 0.17%. 500명 샘플에서는 기대값이 0.4건이라 아예 안 나올 수 있다

14.2 재현성과 증분 실증

`seed: 20260803` + `emit_manifest: true` → `output/manifest.json` 에 시드 · config 해시 · 행수 · commit을 남긴다. 같은 시드 + 같은 config = 같은 데이터.

`incremental_batch` — 마지막 7일치를 2차 배치로 분리해서 증분 적재를 실증한다. `include_updates: true`

— *INSERT만 있으면 증분의 절반만 시험한 것이다.* `nickname_change` · `unfriend` · `report_review` ·

`post_delete` 네 종류의 UPDATE를 섞는다.

15. 판본 이력 v1 → v5

판본	행수	무엇이 깨졌고 무엇을 고쳤나
v1	1억 4,126만	성장 곡선 첫 구현. 결함 8건 을 드러낸 판본
v2	1억 8,826만	전면 감사 — 무결성 · 분포 · 그래프 · 행동 전 영역. 결함 3건 발견
v3 폐기	1억 8,918만	감사 3건 수정본. 확정 직전 <code>user_session</code> 결함 이 나와 중단. 측정값은 남겼다
v4 대체됨	1억 8,922만	목표 지표 10개 전부 달성 · 정합성 17종 · 이상치 9종 위반 0. 확정했다가 정밀 EDA에서 결함 1건 · 경계 5건 이 나왔다
v5 확정	1억 2,370만	요구사항 7가지 전부 충족 — 로그정규 · 요일패턴 · 세션 포함관계 · 로그노이즈 · 탈퇴 종점 · 시간 일관성 · 좋아요 60%. v4에서 "할 수 없다" 15종 중 9종이 열렸다

행이 35% 줄어든 것은 줄어든 것이 아니라 없어야 했던 것이 빠진 것이다

v4에는 하루에 **406.8시간** 투표한 유저-일이 있었다. 시간 예산(10.2절)이 그것을 잘라냈다.

15.1 v5 목표 지표 판정

지표	목표	v4	v5	판정
친구 5명 해금률	90~95%	92.7%	92.6%	통과
힌트 구매 유저	70~80%	77.6%	76.1%	통과
받은 투표 0건	소수	6.3%	6.6%	통과
상위 10% 지목 점유	49%	49.3%	49.8%	통과
완료율 두 측정 차	일치	0.4%p	0.4%p	통과
민감 질문 신고 배수	2배+	3.5배	3.8배	통과
같은 반 친구 0~3명	15% 미만	14.7%	14.7%	통과
접속 세션 시간대 리듬	—	5.3배	3.1배	약해짐

⚠ 시간대 대비가 5.3배 → 3.1배로 약해진 것은 **세션 병합의 부작용**이다 — 30분 안에 이어지는 행동을 한 세션으로 묶으니, v4에서 저녁에 5개로 세지던 것이 1개가 된다. **봉우리 시각(21시)**과 모양은 그대로다.

16. 아홉 번 밟은 함정

"설정에 값이 있어도 코드가 안 읽으면 없는 것"

2026-08-03 ~ 05 사흘 동안 **아홉 번** 밟았다. 전부 오류가 안 나고 데이터를 다 만든다. 분석해야 드러난다.

죽어 있던 설정	증상
<code>growth</code>	성장 곡선이 통째로 무시됨

<code>schools.tiers</code> · <code>schools.adoption</code>	학교 50곳이 전부 첫날 열렸고 정원이 안 지켜졌다
<code>retention.reactivation_rate</code>	복귀 유저가 0명
<code>popularity.top10_share</code>	목표치가 목표로 안 쓰임
Q23 시간대	0~23시가 전부 4.1~4.2% (완전 균등)
<code>behavior</code>	블록을 넣다가 <code>seasonality</code> 를 밖으로 밀어냈다
<code>sessions.weekend_multiplier</code>	요일 효과가 아예 없었다 (변동계수 0.024) — v4 확정 EDA에서 발견
<code>sessions.hour_weights</code>	

구조적으로 막은 방법 세 가지

1. `load_config()` 의 절 복사 방식을 뒤집었다 — 명시적으로 나열한 절만 복사하던 것을, 빠뜨리면 드러나게 바꿨다.
2. 값을 코드 옆 상수로 옮기고 설정으로는 켜고 끄기만 한다 (`DOW_WEIGHTS` + `use_dow_weights`). 값이 설정에만 있으면 또 조용히 죽는다.
3. 죽은 설정은 지운다 — `weekend_multiplier` · `hour_weights` 를 삭제해서 다시는 안 읽히는 값이 남지 않게 했다.

교훈 — 설정 파일은 선언이지 보증이 아니다. 값이 있다는 것과 그 값이 결과에 도달했다는 것은 다른 얘기이고, 둘의 차이는 오류로 나타나지 않는다. 이 프로젝트의 `doc_lint` · 정합성 17종 · 분포 리포트가 전부 같은 이유로 있다.

17. 이 데이터로 할 수 없는 것 7가지

v4에서는 15가지였다. v5가 9종을 열었다 — 퍼널 · 체류시간 분포 · 이탈 예측 · 요일 효과 · 상위 1% 세그먼트 · 집중 지표 절대값 · 접속 세션 기준 복귀 · 세션 길이와 활동의 관계 · 중복 로그 정제 연습.

여전히 불가능	이유
질문별 인기 · A/B	출제량 차이가 설계된 것이 아니라 표본 노이즈
후보 슬롯 위치 편향	변동계수 0.0039 — 검정하면 반드시 기각
문항 위치별 응답 품질	변동계수 0.0002
성별 상호작용 (동성 선호)	지목이 인구 비중과 같다
게시판 카테고리별 차이	변동계수 0.0046
앱 버전 · 플랫폼 · 기기 단위	버전 매달 33% 고정 · 기기 1개가 플랫폼 3개를 오간다
광고 네트워크 · 시청시간 분포	admob 하나뿐 · 시청시간이 균등 (v5에서 빠뜨렸다)

17.1 주의해야 할 것 셋

1. 중복 로그(0.4%)를 안 걸러내면 체류시간이 부풀려진다.
2. 시간대 히트맵을 세션 수로만 그리면 대비가 약하다 — 세션 병합 탓 (5.3배 → 3.1배).
3. BAN 코호트는 시각으로 잘라야 한다 — 34명이 제재 후에도 투표한다 (탈퇴만 고쳤다).

이 장이 이 문서에서 제일 중요하다

합성 데이터를 만든 사람은 많다. 그 데이터로 무엇을 못 하는지 변동계수까지 재서 적어 둔 경우는 드물다.

분석을 시작하기 전에 이 장을 먼저 본다. 여기 없는 질문을 던지면 데이터가 답을 주지만 그 답은 우연이다.

ping-v2 · 합성 데이터 생성 명세서 v5 · 원본: `generator/config/synthetic-v2.yaml` · `generator/generate.py` · `data/personas.json`

관련 문서 — 결정 이력 `synthetic-data-decision-history.pdf` (30문답) · 검수 `EDA-final-12m-v5.pdf` (23쪽) · 운영 `docs/ops/ops-synthetic-data.md`